DAW · PROYECTO INTERMODULAR

MEMORIA TÉCNICA

Recetario planificación semanal inteligente de comidas

Introducción, objetivos, elección de tecnologías y planificación del proyecto de desarrollo de aplicaciones web.

Roberto de la Casa Sotillo
Ciclo Formativo de Grado Superior · DAW

Curso 2026/2027
Entregas 1 & 2

01 Introducción

> Recetario es una aplicación web que guarda las recetas del usuario y genera automáticamente su calendario semanal de comidas, adaptado a sus alergias, su dieta y su objetivo calórico.

Organizar una alimentación saludable y variada a lo largo de la semana es una tarea que requiere tiempo y conocimientos nutricionales de los que no todo el mundo dispone. Es habitual recurrir siempre a las mismas recetas por comodidad, o renunciar a cocinar de forma equilibrada por falta de planificación.

Recetario permite a cualquier usuario guardar y organizar sus recetas de cocina, y generar automáticamente un calendario semanal de comidas (desayuno, comida, cena y snack) adaptado a sus necesidades: sus alergias e intolerancias, el tipo de dieta que sigue y su objetivo calórico diario.

A diferencia de un recetario tradicional, que se limita a almacenar y mostrar recetas, Recetario incorpora un **motor de reglas nutricionales propio** que decide automáticamente qué recetas encajan en el plan semanal de cada usuario, evitando ingredientes no permitidos y ajustando el aporte calórico total, sin que el usuario tenga que hacer ese cálculo manualmente.

Arquitectura. Frontend en Vue 3 + TypeScript que consume una API REST en Node.js/Express, base de datos relacional PostgreSQL gestionada mediante Prisma, y despliegue en producción sobre Vercel y Neon.

02 Objetivos del proyecto

Este proyecto tiene como finalidad desarrollar una aplicación web completa de gestión de recetas con generación automática de calendarios de comidas saludables. Para ello se persiguen los siguientes objetivos:

- Diseñar y desarrollar una aplicación web completa de gestión de recetas de cocina.
- Implementar un sistema de autenticación seguro basado en JWT.
- Modelar una base de datos relacional que almacene recetas, usuarios, preferencias nutricionales y calendarios de comidas.
- Desarrollar un motor de reglas que genere automáticamente calendarios semanales de comidas ajustados a las restricciones y objetivos nutricionales de cada usuario.
- Diseñar una interfaz de usuario clara y accesible para gestionar recetas y visualizar el calendario semanal.
- Desplegar la aplicación en un entorno de producción real, con un pipeline de integración y despliegue continuo (CI/CD).
- Garantizar la calidad y seguridad del software mediante pruebas automatizadas y buenas prácticas (OWASP).

03 Objetivos personales

CONSOLIDAR

- Afianzar los conocimientos adquiridos durante el ciclo en frontend, backend y bases de datos, integrándolos en un proyecto real de principio a fin.

PROFUNDIZAR

- Profundizar en el diseño de un motor de reglas / lógica de negocio no trivial, más allá de un CRUD básico.

EXPLORAR

- Aprender a desplegar una aplicación en un entorno serverless (Vercel + Neon), algo no visto en profundidad durante el ciclo.

RESPONSABILIDAD CON IA

- Usar herramientas de IA de forma responsable como apoyo al desarrollo, entendiendo y pudiendo defender cada parte del código entregado.

04 Elección de tecnologías

Para que cualquier persona que lea esta memoria entienda las decisiones técnicas, conviene aclarar antes dos ideas: el **frontend** es la parte de la aplicación que el usuario ve y con la que interactúa en su navegador; el **backend** es la parte que funciona en un servidor, fuera de su vista, gestiona los datos y protege la información.

Frontend — Vue 3

Construye la interfaz visual. Sencillo de aprender, muy documentado, y las pantallas se actualizan solas cuando cambian los datos (el total del calendario se recalcula al añadir una receta, sin recargar la página).

Backend — Node.js + Express

Responde a las peticiones de la aplicación («dame mis recetas», «genera mi calendario»). Permite usar TypeScript tanto en frontend como en backend, simplificando el desarrollo.

TypeScript

Extensión de JavaScript que obliga a definir de antemano el tipo de cada dato. Evita errores típicos y el propio editor avisa antes de ejecutar nada.

Base de datos — PostgreSQL

Guarda de forma organizada y permanente usuarios, recetas, preferencias y calendarios. Gratuita, fiable y adecuada para datos relacionados entre sí.

Prisma (ORM)

Traductor entre el código y la base de datos: los datos se piden con código normal en vez de SQL, y avisa de errores de nombre o tipo antes de que el programa falle.

Seguridad — JWT y bcrypt

JWT es una «pulsera identificativa» digital tras iniciar sesión. bcrypt revuelve las contraseñas de forma irreversible antes de guardarlas en la base de datos.

Validación — Zod

Comprueba que los datos que llegan al servidor tienen el formato esperado antes de guardarlos, evitando datos incorrectos o malintencionados.

Pruebas — Vitest

Programas que comprueban automáticamente que el código sigue funcionando como se espera, detectando errores antes de que lleguen a producción.

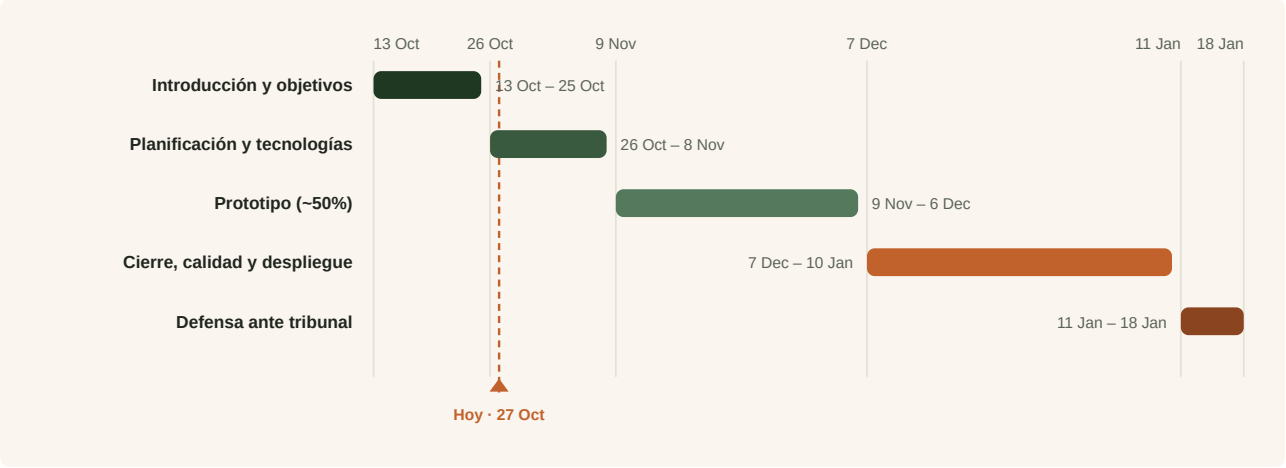
Despliegue — Vercel y Neon · Control de versiones — GitHub y GitHub Actions

Vercel aloja la parte visual y el servidor; Neon hace lo mismo con la base de datos: cada cambio subido se publica solo, sin servidor propio que mantener. GitHub guarda el historial de cambios y GitHub Actions ejecuta las pruebas automáticamente en cada subida, avisando si algo se ha roto.

En resumen, estas tecnologías se han elegido para que el proyecto sea seguro, fácil de mantener y pueda publicarse en internet sin coste ni infraestructura propia, dejando el esfuerzo de desarrollo centrado en la parte más importante: el motor de reglas que genera los calendarios de comidas.

05 Planificación

El proyecto se organiza en cinco fases con fecha de entrega fija marcada por el tutor, gestionadas mediante un tablero Kanban en GitHub (no sprints cerrados, al ser un desarrollo individual sin equipo con el que coordinar iteraciones).



CRONOGRAMA DEL PROYECTO · CINCO FASES · OCTUBRE 2026 – ENERO 2027

FASE	PERIODO	DURACIÓN	QUÉ INCLUYE
Introducción y objetivos	13–25 oct	13 días	Backlog de historias de usuario, elección del stack tecnológico, apartados 1-2 de la memoria
Planificación y tecnologías	26 oct – 8 nov	14 días	Diagrama entidad-relación, diagrama de arquitectura, diagrama de flujo funcional, cronograma, mockups, esqueleto desplegado
Prototipo (~50%)	9 nov – 6 dic	28 días	Autenticación, CRUD de recetas, preferencias nutricionales, motor de reglas, diseño del protocolo de pruebas
Cierre, calidad y despliegue	7 dic – 10 ene	35 días	Calendario semanal editable, tests automatizados, despliegue final en producción, resto de apartados de la memoria
Defensa ante tribunal	11–18 ene	8 días	Ensayo cronometrado de la exposición oral

El desglose completo en tareas, con las etiquetas de epic y prioridad, está en el tablero Kanban del proyecto y en los hitos del repositorio, donde cada fase es un hito con su fecha límite y las tareas (issues) asignadas.

06 Control de versiones

El código se versiona con Git y se aloja en un repositorio público de GitHub. Al ser un desarrollo individual, el flujo de trabajo es sencillo: cada historia de usuario se desarrolla en su propia rama (`feature/us-XX-descripcion`), con commits pequeños y descriptivos, y se integra en `main` mediante un pull request, lo que deja un historial organizado y revisable aunque no haya otra persona revisando.

El seguimiento del trabajo se hace con GitHub Issues (una historia de usuario = un issue) organizadas en el tablero Kanban (Backlog → Lista para hacer → En progreso → En revisión/test → Hecho) y agrupadas en hitos (milestones) que corresponden a las fechas de entrega de la tabla anterior. Cada cambio que se sube a `main` dispara automáticamente el pipeline de GitHub Actions (lint + tests), de modo que un fallo se detecta antes de llegar a producción.

Flujo de trabajo

1 · Rama por historia

`feature/us-XX-descripcion` a partir de `main`, un issue por historia de usuario.

2 · Commits y PR

Commits pequeños y descriptivos; integración en `main` mediante pull request.

3 · CI automática

GitHub Actions ejecuta lint + tests en cada subida a `main`.

4 · Tablero Kanban

Backlog → Lista para hacer → En progreso → En revisión/test → Hecho.